

DAY 7 — INTERVIEW QUESTIONS

Hashing + Intervals • Backend Reliability • Testing • Concurrency • System Design

Detailed textbook-style interview preparation

Day 7 Interview Questions

Absolutely. For Day 7, the interview preparation should test whether you can explain the concepts you actually studied and the architecture you actually built—not just recite definitions.

Day 7 Topic Coverage

DSA

- Hashing
- Hash Map / Hash Set
- Canonical representation
- Group Anagrams
- Longest Consecutive Sequence
- Intervals
- Merge Intervals
- Insert Interval
- Complexity and pattern recognition

Backend / Database

- PostgreSQL failure handling
- Transactions and rollback
- Database connection lifecycle
- Connection pooling
- SQL parameterization
- Repository architecture
- Concurrency
- Lost updates
- Optimistic concurrency

Testing

- Failure-oriented testing
- Database integration testing
- Test isolation
- Integrity constraint failures

System Design

- Scaling the document service
- Database bottlenecks
- Reliability
- Observability

PART I — DSA INTERVIEW QUESTIONS

A. Hashing Fundamentals

Q1. What is hashing?

Explain: • hash function • hash table • key/value • average lookup complexity • collisions

Q2. What is the difference between a HashMap and a HashSet?

Be able to explain: HashMap key → value versus: HashSet value → membership Then answer: When would you choose one over the other?

Q3. What is the average time complexity of lookup in a hash table?

Expected discussion: Average: $O(1)$ Worst case: $O(n)$ Do not stop at $O(1)$. Explain why the worst case can occur.

Q4. What causes collisions in a hash table?

Explain what happens when: $\text{hash}(\text{key1}) == \text{hash}(\text{key2})$ and how hash-table implementations deal with collisions.

Q5. Why are hash tables useful in algorithmic problems?

Give examples where hashing converts: $O(n^2)$ into approximately: $O(n)$

B. Group Anagrams

Q6. How would you solve Group Anagrams?

Start with the problem rather than immediately giving code. Input → identify equivalence → canonical representation → HashMap → groups

Q7. What makes two strings anagrams?

For example: eat tea ate Why are they equivalent?

Q8. What can you use as the key in Group Anagrams?

Discuss at least: Approach 1 Sorted string: eat → aet tea → aet ate → aet Approach 2 Character-frequency representation. Explain the trade-off.

Q9. What is the complexity of the sorting-based solution?

Suppose there are: N strings maximum string length K Explain the complexity rather than simply memorizing it.

Q10. Can Group Anagrams be solved without sorting each string?

If yes: How? Discuss character frequency counting.

Q11. What edge cases exist?

Consider: [] [""], ["a"], ["ab", "ba"], ["abc", "def"]

Q12. Google follow-up

What if the strings contain Unicode characters? What assumptions does your character-frequency solution make?

C. Longest Consecutive Sequence

Q13. What is the brute-force approach?

Explain why sorting is a natural first solution.

Q14. What is the complexity of the sorting solution?

Why is it: $O(n \log n)$ rather than: $O(n)$?

Q15. How can Longest Consecutive Sequence be solved in expected $O(n)$?

Explain the HashSet approach. The important structure is: numbers $\downarrow$ HashSet $\downarrow$ find sequence starts $\downarrow$ expand

Q16. What is a sequence start?

For: 1, 2, 3, 4 why is 1 a sequence start but 2 isn't?

Q17. Why do we check whether num - 1 exists?

Explain why: if num - 1 not in nums: identifies a potential beginning.

Q18. Why don't we expand from every number?

This is an important complexity question. Explain how starting only from sequence beginnings prevents repeated work.

Q19. What happens with duplicates?

Example: [100, 4, 200, 1, 3, 2, 2] How does your HashSet-based solution behave?

Q20. What are the space and time complexities?

Explain: Time: Expected $O(n)$ Space: $O(n)$ and why the "expected" qualification matters.

D. Intervals

Q21. What clues tell you that a problem is an Interval problem?

Examples: • ranges • start/end • overlapping periods • meetings • schedules • bookings • time windows

Q22. Why do we usually sort intervals?

Explain why sorting by start time simplifies the problem.

Q23. What is the overlap condition?

For: [current_start, current_end] [next_start, next_end] when do they overlap?

Q24. What invariant do you maintain while merging intervals?

This is one of the most important Day 7 questions. You should be able to explain what the current merged interval represents at every iteration.

E. Merge Intervals

Q25. Solve Merge Intervals.

Walk through: [1,3] [2,6] [8,10] [9,12] step by step. Expected: [1,6] [8,12]

Q26. Why is Merge Intervals $O(n \log n)$?

Break the complexity into: sorting + linear scan

Q27. What happens when the next interval overlaps?

Explain how you update: current_end

Q28. What happens when the next interval does not overlap?

Explain why the current interval can be finalized.

Q29. What if intervals are already sorted?

Can the complexity improve? What part of the algorithm changes?

Q30. What if intervals are: [1,4] [4,5] Do they overlap?

Your answer should explicitly state the boundary convention you're using.

Q31. What if an interval completely contains another?

Example: [1,10] [2,5] What should happen?

F. Insert Interval

Q32. Explain Insert Interval.

Think in three regions: before overlap overlap region after overlap

Q33. What is different between Insert Interval and Merge Intervals?

Explain the additional reasoning introduced by an already-existing sorted set of intervals plus one new interval.

Q34. What are the edge cases?

Consider: $[]$ $[[1,3]]$ new = $[2,5]$ new interval before everything new interval after everything new interval contained inside an existing interval new interval containing several existing intervals

PART II — BACKEND INTERVIEW QUESTIONS

A. Request Lifecycle

Q35. Walk me through a request to GET /documents/{document_id} in your Project 1.

You should be able to describe: HTTP request ↓ FastAPI route ↓ validation ↓ DocumentService ↓ DocumentRepository ↓ database connection ↓ PostgreSQL ↓ result ↓ response Do this without opening the code.

Q36. Why shouldn't the FastAPI route directly execute SQL?

Discuss separation of: HTTP concerns business logic persistence

Q37. Why have a service layer?

What happens if all business logic is placed directly inside the route?

Q38. Why have a repository layer?

What advantage does this give you if PostgreSQL is replaced later?

B. Database Failure

Q39. What happens if PostgreSQL becomes unavailable while processing a request?

Walk through: API → service → repository → database → exception Then explain how the application should respond.

Q40. Should the API expose the raw PostgreSQL exception to the client?

Why or why not? Think about: • security • abstraction • debugging • API contracts

Q41. Where should database exceptions be translated into application-level errors?

Discuss the boundaries in your architecture.

C. Connection Lifecycle

Q42. Explain your database connection lifecycle.

Your explanation should include: open ↓ use ↓ commit / rollback ↓ cleanup ↓ close

Q43. Why is finally important for database resources?

What happens if an exception occurs before normal cleanup?

Q44. Why should failed database operations be rolled back?

Give a concrete example.

Q45. What is the difference between COMMIT and ROLLBACK?

D. Connection Pooling

Q46. Why can creating a new database connection for every request be expensive?

Discuss: • network connection • authentication • database resources • latency

Q47. What is connection pooling?

Explain: Request ↓ Acquire connection ↓ Execute query ↓ Release connection

Q48. What happens if the connection pool is exhausted?

This is an important production question. Discuss: • waiting • timeout • rejected requests • backpressure

Q49. How would you determine whether database connection saturation is occurring?

What metrics would you monitor?

E. SQL Security

Q50. Why is this dangerous?

```
f"SELECT * FROM documents WHERE id = '{document_id}'"
```

Q51. What is SQL injection?

Explain the underlying mechanism rather than just saying "it's a security vulnerability."

Q52. Why is parameterized SQL safer?

For example: `cursor.execute("SELECT ... WHERE id = %s", (document_id,))`

PART III — CONCURRENCY

A. Lost Updates

Q53. What is a lost update?

Suppose: Document version = 1 Two requests read it simultaneously: Request A → reads 1 Request B → reads 1
Then: A → writes 2 B → writes 3 Explain the race.

Q54. Why can concurrent updates cause incorrect results?

Explain the read-modify-write sequence.

Q55. What is optimistic concurrency control?

Explain the idea of: expected version + conditional update

Q56. Explain this query:

UPDATE documents SET version = ... WHERE id = ... AND version = expected_version; What does zero affected rows mean?

Q57. When would optimistic concurrency be useful?

Relate the answer to your document service.

Q58. What are other approaches to concurrency control?

At a high level discuss: • optimistic locking • pessimistic locking • transactions • database isolation You do not need to implement all of them on Day 7.

PART IV — TESTING

A. Integration Testing

Q59. Why do you need PostgreSQL integration tests if unit tests already exist?

Explain what integration tests validate that unit tests cannot.

Q60. What does a PostgreSQL integration test actually test?

Discuss the chain: API → service → repository → connection → PostgreSQL

Q61. Why is test isolation important?

Imagine: Test A creates document A Test B expects database to be empty What can go wrong?

Q62. How can database tests be isolated?

Discuss approaches such as: TRUNCATE transactions rollback dedicated test database containers You don't need to implement every method.

Q63. What is the difference between a unit test and an integration test?

Use your Project 1 as the example.

B. Failure Testing

Q64. Why shouldn't you test only successful CRUD operations?

Explain the importance of: failure paths

Q65. How would you test PostgreSQL unavailability?

Describe a test strategy without fabricating an implementation or result.

Q66. How would you test duplicate primary-key behavior?

Walk through: reproduce → observe → identify exception → decide translation boundary → implement if necessary

Q67. What should happen if deleting a nonexistent document?

Why should the behavior be consistent with your API contract?

PART V — SYSTEM DESIGN

Document Service at 10× Scale

Q68. Your document service receives 10× the current traffic. What breaks first?

Don't immediately say "add Kubernetes." Start with: measure → identify bottleneck → scale bottleneck

Q69. How would you horizontally scale FastAPI?

Explain: Load Balancer ↓ FastAPI instance 1 FastAPI instance 2 FastAPI instance 3 Why does this work?

Q70. What becomes the bottleneck after scaling API servers?

Think about: PostgreSQL connection pool CPU I/O network

Q71. How would you scale PostgreSQL reads?

Discuss conceptually: indexes caching read replicas and explain the trade-offs.

Q72. What would you cache in a document service?

Potential candidates: document metadata frequently accessed documents Then ask: What happens when the underlying document changes?

Q73. What consistency problem does caching introduce?

Discuss stale data.

Q74. How would you make the document service resilient to database failures?

Think about: timeouts connection pools health checks retries backpressure fallback behavior monitoring Do not blindly recommend retries. Ask: Is the operation safe to retry?

Q75. Why can retries make an outage worse?

Discuss: database overloaded ↓ requests retry ↓ more database traffic ↓ database becomes even more overloaded This is the beginning of understanding retry storms.

PART VI — OBSERVABILITY

Q76. What should you monitor for your document service?

At minimum think about: request latency error rate throughput database latency connection pool utilization

Q77. What is the difference between logs, metrics and traces?

Explain each using Project 1.

Q78. If GET /documents/{id} suddenly becomes slow, how would you debug it?

Walk through: request latency ↓ application logs ↓ trace ↓ repository timing ↓ database query ↓ database metrics

Q79. What metrics would tell you PostgreSQL is becoming the bottleneck?

Examples: • query latency • connection utilization • CPU • I/O • lock contention Explain why each metric matters.

PART VII — PROJECT-SPECIFIC GOOGLE QUESTIONS

Q80. Why did you move from an in-memory dictionary to PostgreSQL?

Don't answer: "Because PostgreSQL is production-ready." Explain the engineering requirements: durability persistence concurrency querying constraints multi-process access

Q81. Why didn't you change your API when changing the persistence layer?

This tests whether you understand abstraction.

Q82. Suppose your PostgreSQL repository needs to be replaced by a mock repository during a unit test. How would your architecture support that?

Discuss dependency boundaries.

Q83. Suppose PostgreSQL is replaced by another relational database. Which layer should change?

Which layers should ideally remain unchanged?

Q84. Suppose document storage grows from 1,000 documents to 100 million documents. What parts of your current design would you revisit?

Discuss: indexes partitioning query patterns storage connection pooling read scaling caching archival observability

Q85. What is the biggest limitation of your current document service?

This is a very important interview question. Give an honest limitation based on the actual implementation. Do not invent a production capability you have not built.

PART VIII — CROSS-DOMAIN QUESTIONS

Q86. Your API works locally, but PostgreSQL becomes slow in production. How would you investigate?

Q87. Your API latency increases but PostgreSQL query latency does not. Where would you investigate next?

Q88. PostgreSQL latency is low, but API latency is high. What could explain that?

Q89. Your connection pool is exhausted but CPU usage is low. What could be happening?

Q90. A request fails after modifying the database. How do you ensure the transaction does not leave partial state?

Q91. Two users update the same document simultaneously. How should your system behave?

Q92. Your service is horizontally scaled to ten instances. What assumptions about the in-memory repository are no longer valid?

This is a particularly useful question because it connects your old architecture to the new PostgreSQL architecture.

Q93. Why is persistent storage necessary when running multiple API instances?

Q94. What happens if one API instance crashes after PostgreSQL has committed a document?

Why is durable storage important here?

PART IX — RAPID-FIRE ROUND

Try answering each in 30 seconds or less.

- 1. What is a HashSet?
- 2. HashMap vs HashSet?
- 3. Average hash lookup complexity?
- 4. Worst-case hash lookup complexity?
- 5. What is a canonical representation?
- 6. Why use sorting in Group Anagrams?
- 7. Alternative to sorting?
- 8. Why use a set in Longest Consecutive Sequence?
- 9. What is a sequence start?
- 10. Why sort intervals?
- 11. What is interval overlap?
- 12. Merge Intervals complexity?
- 13. What is a transaction?
- 14. COMMIT vs ROLLBACK?
- 15. Why use finally?
- 16. What is connection pooling?
- 17. What is SQL injection?
- 18. Why parameterized queries?
- 19. What is a lost update?
- 20. What is optimistic concurrency?
- 21. What is test isolation?
- 22. Unit test vs integration test?
- 23. What happens when PostgreSQL is unavailable?
- 24. What would you monitor?
- 25. Why use a repository layer?

PART X — GOOGLE-STYLE MOCK INTERVIEW

Round 1 — Coding

Problem:

Given an array of integers, return the length of the longest consecutive sequence.

You have: 30 minutes.

You must explain:

Problem clarification ↓ Brute force ↓ Complexity ↓ Optimization ↓ HashSet ↓ Invariant ↓ Code ↓
Test cases ↓ Complexity

Round 2 — Coding

Problem:

Given a collection of intervals, merge all overlapping intervals.

You have: 30 minutes.

Before coding, explicitly state:

Pattern: Sort + Greedy

Then explain the invariant.

Round 3 — Backend

Interviewer:

“Walk me through your document service.”

```

Client  ↓  FastAPI
↓
Service  ↓
Repository  ↓  PostgreSQL

```

Then answer:

“What happens if PostgreSQL fails?”

Round 4 — Concurrency

Interviewer:

“Two requests update the same document simultaneously. How does your system prevent one update from silently overwriting the other?”

Explain optimistic concurrency.

Then answer:

“Why might you choose optimistic rather than pessimistic locking?”

Round 5 — Scaling

Interviewer:

“Your document service now receives 10× traffic. Design the next version.”

Requirements ↓ Scale ↓ APIs ↓ Data model ↓ Architecture ↓ Bottlenecks ↓ Caching ↓ Partitioning ↓
Consistency ↓ Reliability ↓ Monitoring ↓ Cost

Do not jump straight to technologies.

DAY 7 INTERVIEW DEFINITION OF DONE

You should not mark interview preparation complete simply because you read these questions.

DSA

- Explain HashMap vs HashSet
- Recognize hashing problems
- Solve Group Anagrams
- Solve Longest Consecutive Sequence
- Explain why the HashSet approach is expected $O(n)$
- Recognize interval problems
- Solve Merge Intervals
- Explain the merge invariant
- Explain Insert Interval
- Explain all major edge cases

Backend

- Explain your complete request lifecycle
- Explain why you use Service + Repository
- Explain PostgreSQL failure behavior
- Explain connection lifecycle
- Explain rollback
- Explain connection pooling
- Explain SQL injection
- Explain parameterized queries
- Explain lost updates
- Explain optimistic concurrency

Testing

- Explain unit vs integration testing
- Explain database test isolation
- Explain failure-oriented testing
- Explain duplicate/integrity failures
- Explain how you would test database unavailability

System Design

- Explain how to scale the document service
- Identify database bottlenecks
- Discuss caching trade-offs
- Discuss connection-pool saturation
- Discuss retries and retry storms
- Discuss observability
- Explain your current architectural limitations honestly

Most important interview habit for Day 7

For every answer, practice this sequence:

What → Why → How → Trade-off → Failure mode

For example, don't answer:

“Connection pooling reuses connections.”

Instead:

What: Connection pooling maintains reusable database connections.

Why: Creating connections repeatedly adds latency and resource overhead.

How: A request acquires a connection, performs its database work, and returns the connection to the pool.

Trade-off: The pool consumes resources and has a finite capacity.

Failure mode: If all connections are occupied, requests may wait or time out.

That style is much closer to the reasoning expected in a Google SWE interview than definition-only preparation.